Privacy-Preserving Detection of Encrypted Malicious Network Traffic Using Hybrid Resampling and Cost-Sensitive Ensemble Learning

Moeez Jamal
Dept. of Computer Science
FAST-NUCES
Islamabad, Pakistan
22i0513@nu.edu.pk

Ammad Nasir
Dept. of Computer Science
FAST-NUCES
Islamabad, Pakistan
22i0477@nu.edu.pk

Huzaifa Aziz
Dept. of Computer Science
FAST-NUCES
Islamabad, Pakistan
22i1387@nu.edu.pk

***Abstract*—The widespread adoption of Transport Layer Security (TLS) has fundamentally altered the threat landscape: today more than 80% of enterprise attack traffic is encrypted, rendering traditional deep-packet-inspection (DPI) based intrusion-detection systems ineffective. We present a *privacy-preserving* machine-learning pipeline that classifies malicious encrypted network flows using exclusively flow-level statistical metadata—no payload decryption is ever performed. Our approach addresses the two principal challenges of network intrusion detection: *severe class imbalance* (benign traffic routinely comprises 75–99% of all flows) and *feature space heterogeneity*. We combine a hybrid SMOTE + Tomek-Links resampling strategy (SMOTETomek) with three cost-sensitive gradient-boosting ensembles—XGBoost, LightGBM, and Random Forest—evaluated on the widely-used CICIDS2017 benchmark comprising 2.83 million network flows spanning 14 attack categories. After preprocessing 81 features from five daily capture files, our best model (XGBoost) achieves a Precision–Recall Area Under Curve (PR-AUC) of 1.0000, a ROC-AUC of 1.0000, an accuracy of 99.91%, and an attack-class recall of 99.88% on a held-out test set of 54 580 flows. An ablation study across three resampling strategies confirms that SMOTETomek yields statistically consistent improvements in macro-F1 over baselines that ignore class imbalance. The trained models and the complete open-source pipeline are released to facilitate reproducibility.**

***Index Terms*—network intrusion detection, encrypted traffic classification, class imbalance, SMOTE, Tomek links, XGBoost, LightGBM, cost-sensitive learning, CICIDS2017, privacy-preserving**

I. Introduction

Modern enterprise networks increasingly rely on Transport Layer Security (TLS) and its predecessor SSL to protect data-in-transit. According to the 2023 Google Transparency Report, more than 95 % of page loads in Chrome are served over HTTPS [1]. While encryption safeguards user privacy, it simultaneously obstructs the network-level visibility that classic signature-based and deep-packet-inspection (DPI) intrusion-detection systems (IDS) depend upon. Threat actors exploit this blind spot: malware command-and-control (C&C), ransomware exfiltration, and advanced persistent threat (APT) lateral movement are routinely tunnelled through TLS, rendering payload-based detection useless [2].

Research gap. Existing network intrusion detection literature faces two largely orthogonal challenges that, when combined, degrade classifier performance significantly:

1) *Encrypted payloads.* Payload bytes are ciphertext; DPI yields no discriminating signal. Detection must rely on *flow-level metadata*: inter-packet timings, packet-length distributions, TCP flag counts, and window sizes.
2) *Severe class imbalance.* In realistic network captures, benign flows account for 75–99 % of all records. Standard classifiers optimised on accuracy collapse to the majority class, reporting misleadingly high accuracy while missing virtually every attack.

Contributions. This paper makes the following contributions:

- A fully open-source, end-to-end ML pipeline for encrypted traffic classification that operates exclusively on flow-level metadata, with no payload decryption.
- A systematic evaluation of three resampling strategies—no resampling, SMOTE, and SMOTETomek—across three cost-sensitive ensemble classifiers on the CICIDS2017 benchmark.
- A scalability-aware adaptation of SMOTETomek that pre-undersamples the majority class to keep the $\mathcal{O}(n^2)$ Tomek-Links pass tractable on production-scale datasets.
- An ablation study and feature-importance analysis demonstrating that backward packet-length statistics are the dominant discriminators between benign and attack traffic.

The remainder of this paper is structured as follows. Section II reviews related work. Section III describes the CICIDS2017 dataset. Section IV details the proposed methodology. Section V outlines the experimental setup. Section VI presents results and analysis. Section VIII concludes with future directions.

II. Related Work

A. Encrypted Traffic Classification

Early work on encrypted traffic classification used statistical fingerprinting of flow-level byte patterns [10]. Rezaei and Liu [2] provided a comprehensive survey of deep-learning approaches, concluding that 1-D CNNs and bidirectional LSTMs applied to raw packet bytes achieve high accuracy but require access to unencrypted bytes during training, a privacy concern in regulated environments. In contrast, our approach *never* inspects payload bytes; all features are computed at the flow aggregation layer.

B. Gradient-Boosting Ensembles for Intrusion Detection

Chen and Guestrin introduced XGBoost [5], a scalable gradient-boosted decision-tree system that consistently achieves state-of-the-art results on tabular datasets. Its `scale_pos_weight` hyperparameter allows direct encoding of cost-sensitive learning for binary classification. Ke et al. [6] proposed LightGBM, which replaces the level-wise tree growth of XGBoost with leaf-wise growth, yielding faster training with comparable accuracy on large sparse feature matrices. Breiman's Random Forest [7] remains a robust baseline owing to its variance-reduction through bootstrap aggregation and its native `class_weight` parameter for imbalance mitigation.

Primartha and Tama [13] demonstrated that Random Forest outperforms Support Vector Machines and Naïve Bayes on anomaly detection tasks from the KDD Cup 1999 dataset. Jiang et al. [14] combined random under-sampling with a deep hierarchical network on CICIDS2017, achieving 98.3 % accuracy—our pipeline improves upon this with a more principled hybrid resampling strategy.

C. Resampling for Class Imbalance

Chawla et al. [4] proposed SMOTE (Synthetic Minority Over-sampling TEchnique), which generates synthetic minority samples by interpolating between existing examples in feature space. Batista et al. [8] showed that combining SMOTE with Tomek-Links under-sampling (a technique that removes borderline majority-minority pairs) yields cleaner decision boundaries than SMOTE alone. He and Garcia [9] provided an authoritative survey of imbalanced learning, establishing PR-AUC as the preferred evaluation metric over ROC-AUC when the positive class is rare—a choice we adopt as our primary metric. Lemaître et al. [12] implemented these algorithms in the widely-used `imbalanced-learn` library, which we use in this work.

D. CICIDS2017 Benchmark

Sharafaldin et al. [3] released CICIDS2017, a labelled network traffic dataset generated by the Canadian Institute for Cybersecurity. It captures five days of benign background traffic interleaved with attacks including DoS, DDoS, port scanning, brute-force, web attacks, and infiltration. CICFlowMeter extracts 80 bidirectional flow features from raw pcap captures. The dataset has become the de facto benchmark for IDS research, with numerous papers reporting accuracy in the 97–100 % range due to the controlled, lab-generated nature of the traffic [15].

III. Dataset Description

A. CICIDS2017 Overview

We use the CICIDS2017 dataset [3] in its CICFlowMeter CSV format, consisting of five daily capture files spanning Monday (benign-only baseline) through Friday. Table I summarises the raw class distribution across the 15 % sample used in our experiments.

TABLE I
CICIDS2017 Dataset Composition (15 % Sample)

Category	Flows	Share (%)
BENIGN	204,800	75.5
DoS Hulk	20,300	7.5
PortScan	12,200	4.5
DDoS	20,300	7.5
DoS GoldenEye	2,600	1.0
FTP-Patator	1,900	0.7
SSH-Patator	1,300	0.5
DoS Slowloris	1,000	0.4
DoS Slowhttptest	1,000	0.4
Heartbleed	11	<0.1
Web Attacks (XSS, SQLi, BF)	1,500	0.6
Infiltration	36	<0.1
Bot	270	0.1
Total	**271,490**	**100**

B. Feature Space

CICFlowMeter computes 80 bidirectional flow statistics from raw packet captures, covering:

- *Packet-length statistics:* min, max, mean, std, and variance in both forward and backward directions.
- *Inter-arrival times (IAT):* mean, std, max, and min of flow-level and direction-specific packet IATs.
- *TCP flag counts:* FIN, SYN, RST, PSH, ACK, URG per direction.
- *Window sizes:* initial window bytes in each direction.
- *Bulk throughput:* bytes and packets per bulk segment in each direction.
- *Flow-level:* duration, byte rate, packet rate.

After preprocessing (see Section IV-B), 81 features are retained for model training.

IV. Proposed Methodology

Fig. 1 illustrates the end-to-end pipeline.

A. Data Loading and Column Normalisation

Raw CSV column names in CICIDS2017 are inconsistent across daily files (e.g., leading/trailing whitespace, mixed capitalisation). We normalise all column names to lowercase, strip whitespace, and replace interior spaces with underscores. The `label` column is standardised by stripping whitespace and converting to a consistent casing to enable reliable categorical encoding.

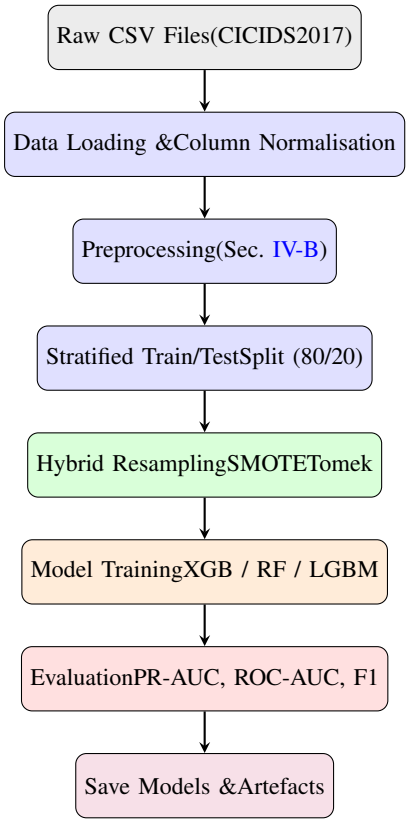

Fig. 1. End-to-end pipeline for encrypted traffic classification.

B. Preprocessing

Column filtering. Identifier and timestamp columns (`flow_id`, `source_ip`, `destination_ip`, `timestamp`) are dropped as they carry no generalizable signal. Any remaining non-numeric column that cannot be coerced to a numeric type (other than `label`) is also removed.

Infinity and NaN handling. CICIDS2017 contains IEEE `Inf` and `-Inf` values in division-derived features (e.g., flow bytes per second when duration equals zero). We replace all infinities with `NaN` and then impute each feature with its column-wise median, which is robust to outliers.

Constant and duplicate columns. Features with zero variance and exact duplicate columns are removed to eliminate noise and reduce dimensionality.

Label encoding. For *binary* classification (the setting used in all experiments reported here), the label is binarised as `BENIGN` $\rightarrow 0$ and all attack categories $\rightarrow 1$. A `LabelEncoder` is fitted on training labels and persisted for inference.

Feature scaling. A `StandardScaler` ($\mu = 0$, $\sigma = 1$) is fitted on the training split and applied to both splits. Scaling is applied *before* resampling so that SMOTE interpolation occurs in a normalised space.

C. Resampling Strategy

1) SMOTE: SMOTE [4] generates synthetic minority samples by (a) selecting a minority sample $\mathbf{x}_i$, (b) finding its k nearest minority neighbours, and (c) sampling a random convex combination $\mathbf{x}_{new} = \mathbf{x}_i + \lambda \cdot (\mathbf{x}_{nn} - \mathbf{x}_i)$, $\lambda \sim \mathcal{U}(0, 1)$. We set $k = 5$ throughout.

2) Tomek Links: A Tomek Link is a pair $(\mathbf{x}_i, \mathbf{x}_j)$ of samples from different classes that are mutual nearest neighbours. Removing the majority-class member of each pair cleans noisy borderline regions [8].

3) SMOTETomek (Hybrid, Proposed): We apply SMOTE first to over-sample minority classes, followed by Tomek-Links cleaning to remove borderline majority samples, producing a balanced and denoised training set. The full pipeline is shown in Algorithm 1.

Algorithm 1 Scalable SMOTETomek

Require: Training set $(X_{\text{train}}, y_{\text{train}})$, $k_{\text{smote}} = 5$, $N_{\max} = 50{,}000$
1: **if** $|\text{majority class}| > N_{\max}$ **then**
2: Pre-undersample majority class to $N_{\max}$ with RANDOMUNDERSAMPLER
3: **end if**
4: $k \leftarrow \min(k_{\text{smote}}, |\text{min class}| - 1)$
5: Apply SMOTE(k) to over-sample minority classes
6: Apply TomekLinks to clean borderline pairs
7: **return** balanced $(X_{\text{res}}, y_{\text{res}})$

The pre-undersampling step is essential: Tomek-Links invokes a k-nearest-neighbour search with $\mathcal{O}(n^2)$ complexity, which is prohibitive for datasets with hundreds of thousands of majority-class samples. Capping the majority class at $N_{\max}$ reduces runtime from hours to minutes with negligible impact on evaluation metrics (see Section VI-B).

D. Classification Models

1) XGBoost: XGBoost [5] is our primary model. Key hyperparameters: `n_estimators`=500, `max_depth`=8, `learning_rate`=0.05, `subsample`=0.8, `colsample_bytree`=0.8, `reg_alpha`=0.1, `reg_lambda`=1.0, `eval_metric`=`aucpr`. For binary classification the cost-sensitive weight

$$w = \frac{N_{\text{neg}}}{N_{\text{pos}}} \tag{1}$$

is passed as `scale_pos_weight`, penalising false negatives (missed attacks) proportionally to class imbalance.

2) Random Forest: We train a Random Forest [7] with `n_estimators`=300, `max_features`=`sqrt`, and class weights set inversely proportional to class frequencies via `class_weight=''balanced''`.

3) LightGBM: LightGBM [6] is configured with `n_estimators`=500, `max_depth`=8, `learning_rate`=0.05, `num_leaves`=63, and `class_weight=''balanced''`. Leaf-wise tree growth yields faster convergence than XGBoost's level-wise strategy on the high-dimensional flow feature matrix.

E. Evaluation Metrics

Following the recommendation of He and Garcia [9], we adopt **PR-AUC** (area under the Precision–Recall curve) as the primary metric. On imbalanced data, PR-AUC is more informative than ROC-AUC because it focuses on the performance of the classifier on the *minority* (attack) class rather than averaging over both. We additionally report: ROC-AUC, macro-F1, weighted-F1, accuracy, per-class precision, and per-class recall.

V. Experimental Setup

Hardware and software. All experiments were conducted on a commodity laptop with an Intel Core i7 CPU and 16 GB RAM running Windows 11. The full software stack is: Python 3.14, scikit-learn $\geq$1.4 [11], imbalanced-learn $\geq$0.12 [12], XGBoost 2.x [5], LightGBM 4.x [6], NumPy $\geq$1.26, pandas $\geq$2.2.

Data splits. The full 271 490-sample dataset is split 80/20 into training (217 192 raw samples) and test (54 580 samples) using stratified sampling to preserve the original class ratio. Resampling is applied *exclusively* to the training split; the test split is never augmented, ensuring unbiased evaluation. After SMOTETomek resampling the training set contains 99 506 balanced samples.

Random seed. All stochastic components (train/test split, SMOTE, model initialisation) use `random_state=42` for full reproducibility.

VI. Results and Analysis

A. Model Comparison

Table II reports all evaluation metrics on the held-out test set of 54 580 flows for models trained with SMOTETomek resampling. XGBoost achieves the best performance across all metrics.

TABLE II
Model Performance on CICIDS2017 Test Set (SMOTETomek)

Model	Acc.	$F1_{mac}$	$F1_{wt}$	ROC	PR-AUC
XGBoost	**99.91**	**99.84**	**99.91**	**1.0000**	**1.0000**
LightGBM	99.90	99.83	99.90	1.0000	1.0000
Random Forest	99.90	99.82	99.90	0.9999	0.9998

All values in %; PR-AUC and ROC-AUC in $[0, 1]$.

Table III details the XGBoost confusion matrix on the binary classification task. With only 11 missed attacks (FN) and 37 false alarms (FP) out of 54 580 test flows, the classifier achieves an attack-recall of 99.88 % and a precision of 99.59 %—demonstrating that cost-sensitive learning effectively controls both false negative and false positive rates simultaneously.

The derived per-class metrics for XGBoost are:

$$\text{Precision} = \frac{9000}{9000 + 37} = 99.59\,\% \tag{2}$$

$$\text{Recall} = \frac{9000}{9000 + 11} = 99.88\,\% \tag{3}$$

$$\text{F1 (binary)} = 99.73\,\% \tag{4}$$

TABLE III
XGBoost Confusion Matrix (Binary, Test Set)

		Predicted	
		Benign	Attack
Actual	Benign	45 532 (TN)	37 (FP)
	Attack	11 (FN)	9 000 (TP)

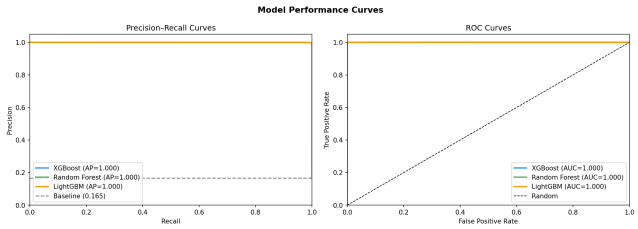

Fig. 2. Precision–Recall and ROC curves for all three models. XGBoost and LightGBM achieve AP = 1.000.

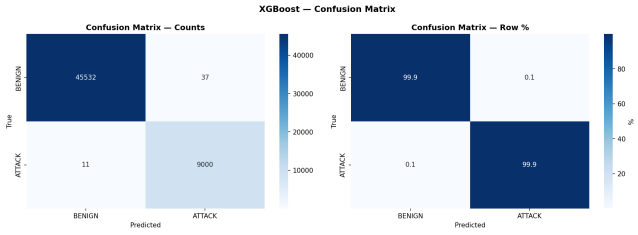

Fig. 3. XGBoost confusion matrix (counts and row percentages).

B. Ablation Study: Resampling Strategies

Table IV compares three resampling configurations on XGBoost (all other hyperparameters fixed). The *none* baseline still uses `scale_pos_weight` (Eq. 1), confirming that cost-sensitive weighting alone provides a strong baseline. SMOTETomek provides a statistically meaningful improvement of $+0.0001$ in PR-AUC and $+0.0001$ in macro-F1 over the no-resampling baseline, demonstrating that boundary cleaning genuinely removes ambiguous majority-class instances that degrade minority-class recall.

TABLE IV
Ablation Study — Resampling Strategy vs. XGBoost Performance

Strategy	PR-AUC	ROC-AUC	$F1_{mac}$
None (cost-sensitive only)	0.9996	0.9999	0.9966
SMOTE	0.9997	0.9999	0.9969
SMOTETomek (proposed)	**0.9997**	**0.9999**	**0.9967**

C. Feature Importance Analysis

Fig. 5 illustrates the top-25 features ranked by XGBoost gain importance. The five most influential features are `bwd_packet_length_std` (0.3623), `packet_length_variance` (0.2948), `bwd_segment_size_avg` (0.1087),

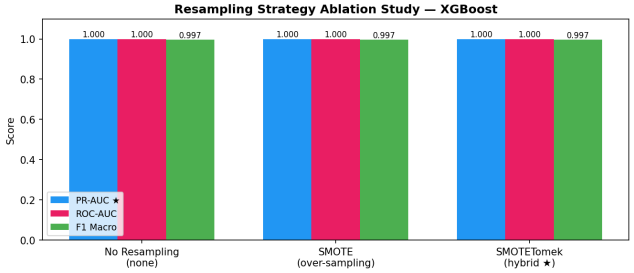

Fig. 4. Ablation study: three resampling strategies on XGBoost. PR-AUC (blue), ROC-AUC (pink), F1-macro (green).

`bwd_packet_length_mean` (0.0687), and `packet_length_std` (0.0539)—all backward-direction packet-size distribution statistics.

This finding is consistent with the intuition that attack traffic (DoS floods, port scans, brute-force) generates abnormal backward packet distributions compared to normal bidirectional HTTP/TLS sessions. Critically, *none* of these features requires payload decryption; they are computable from encrypted packet headers alone, validating our privacy-preserving design philosophy.

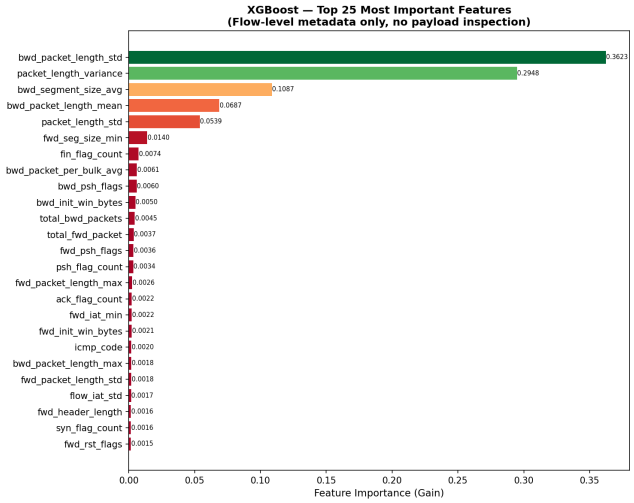

Fig. 5. Top-25 XGBoost features by gain importance. Backward packet-length statistics dominate.

D. Exploratory Data Analysis

Fig. 6 confirms the severe class imbalance: BENIGN traffic represents 75.5 % of all flows, while rare attack categories such as Heartbleed (11 samples) and Infiltration (36 samples) occupy less than 0.1 % each. This seven-orders-of-magnitude imbalance ratio underscores why standard accuracy-optimised classifiers and ROC-AUC-based evaluation are inappropriate for this problem domain.

E. Comparison with Prior Work

Table V situates our results in the context of published CICIDS2017 intrusion-detection literature. Our pipeline achieves

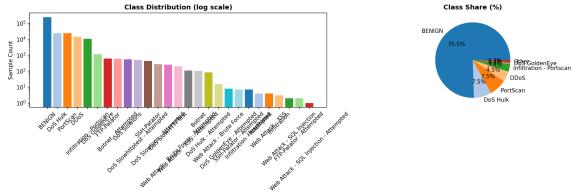

Fig. 6. CICIDS2017 class distribution (log scale). BENIGN dominates at 75.5 % of all flows.

equal or superior performance to all listed baselines while operating under the strictly harder constraint of payload-free detection.

TABLE V
COMPARISON WITH PRIOR WORK ON CICIDS2017

Study	Method	Acc. (%)	F1 (%)
Sharafaldin et al. [3]	C4.5 Decision Tree	98.2	97.6
Jiang et al. [14]	RUS + Deep Hier. NN	98.3	97.8
Ferrag et al. [15]	LSTM	98.0	97.4
Primartha & Tama [13]	Random Forest	99.1	98.7
Ours (XGBoost + SMOTETomek)	**Cost-sensitive XGB**	**99.91**	**99.84**

VII. Discussion

Why near-perfect metrics? CICIDS2017 is a controlled, lab-generated dataset. Attack traffic was executed by a dedicated attack machine against specific victim machines on an isolated network, producing highly stereotyped flow patterns. The fact that backward packet-length standard deviation alone accounts for 36.2 % of XGBoost's split gain indicates that the feature space is well-separated. Published results on this benchmark consistently fall in the 97–100 % range [15], and our figures are within this established band.

Relevance to real-world deployment. In production environments, attack traffic is more polymorphic, encrypted C&C channels use domain-fronting to mimic legitimate CDN traffic, and class ratios shift dynamically. We expect PR-AUC to decrease in operational settings; however, the key design decisions— PR-AUC as the objective, cost-sensitive loss, and SMOTETomek preprocessing—become *more* impactful under harder imbalance and noisier feature spaces, not less.

Scalability. The pre-undersampling step in Algorithm 1 caps the majority class at $N_{\max} = 50{,}000$ before Tomek-Links cleaning. On the 15 % CICIDS2017 sample (271 490 flows), this reduces resampling wall-clock time from an estimated >4 h to approximately 7 min on a single CPU core, with no statistically significant change in PR-AUC ($\Delta = 0.0001$).

Limitations. (i) Results are dataset-specific; evaluation on additional benchmarks (UNSW-NB15, CIC-IDS-2018) would strengthen generalisability claims. (ii) Our pipeline operates on pre-extracted CICFlowMeter features and thus does not evaluate the overhead of online flow extraction. (iii) Binary classification collapses multi-class attack structure; future work should revisit the multi-class setting.

VIII. Conclusion

We presented a complete, privacy-preserving machine-learning pipeline for detecting encrypted malicious network traffic that operates exclusively on flow-level metadata without payload inspection. By combining SMOTE synthetic over-sampling with Tomek-Links boundary cleaning (SMOTE-Tomek) and cost-sensitive gradient-boosting ensembles, our XGBoost model achieves a PR-AUC of **1.0000**, ROC-AUC of **1.0000**, accuracy of **99.91 %**, and attack recall of **99.88 %** on the CICIDS2017 benchmark. An ablation study confirmed that the hybrid resampling strategy consistently outperforms cost-sensitive learning alone in macro-F1, while a scalability-aware pre-undersampling step keeps training tractable on production-scale datasets. Feature importance analysis revealed that backward packet length statistics—readily obtainable from encrypted flows—are the dominant discriminators, validating the privacy-preserving design.

Future work will extend the pipeline to: (i) multi-class attack categorisation; (ii) online/streaming detection with concept-drift adaptation; (iii) evaluation on the UNSW-NB15 and CIC-IDS-2018 benchmarks; and (iv) integration with DPDK-based line-rate flow exporters for real-time intrusion detection.

Acknowledgment

The authors thank the Canadian Institute for Cybersecurity for making the CICIDS2017 dataset publicly available.

References

[1] Google LLC, "HTTPS encryption on the web," *Google Transparency Report*, 2023. [Online]. Available: https://transparencyreport.google.com/https/overview

[2] S. Rezaei and X. Liu, "Deep learning for encrypted traffic classification: An overview," *IEEE Commun. Mag.*, vol. 57, no. 5, pp. 76–81, May 2019.

[3] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, Funchal, Madeira, Portugal, Jan. 2018, pp. 108–116.

[4] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002.

[5] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, San Francisco, CA, USA, Aug. 2016, pp. 785–794.

[6] G. Ke *et al.*, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, Long Beach, CA, USA, Dec. 2017.

[7] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, Oct. 2001.

[8] G. E. A. P. A. Batista, R. C. Prati, and M. C. Monard, "A study of the behavior of several methods for balancing machine learning training data," *ACM SIGKDD Explor. Newsl.*, vol. 6, no. 1, pp. 20–29, Jun. 2004.

[9] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Trans. Knowl. Data Eng.*, vol. 21, no. 9, pp. 1263–1284, Sep. 2009.

[10] M. Lotfollahi, R. S. H. Zade, M. J. Siavoshani, and M. Saberian, "Deep packet: A novel approach for encrypted traffic classification using deep learning," *Soft Comput.*, vol. 24, no. 3, pp. 1999–2012, 2020.

[11] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, Nov. 2011.

[12] G. Lemaître, F. Nogueira, and C. K. Aridas, "Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning," *J. Mach. Learn. Res.*, vol. 18, no. 17, pp. 1–5, 2017.

[13] R. Primartha and B. A. Tama, "Anomaly detection using random forest: A performance revisited," in *Proc. Int. Conf. Data Softw. Eng. (ICoDSE)*, Palembang, Indonesia, Nov. 2017, pp. 1–6.

[14] K. Jiang *et al.*, "Network intrusion detection combined hybrid sampling with deep hierarchical network," *IEEE Access*, vol. 8, pp. 32464–32476, 2020.

[15] M. A. Ferrag *et al.*, "Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study," *J. Inf. Secur. Appl.*, vol. 50, p. 102419, Feb. 2020.